Clasificación automática de emociones en textos en inglés mediante una red neuronal LSTM bidireccional integrada en una aplicación web

Miguel Andres Gomez Gutierrez

Programa de Ingeniería de Sistemas, Corporación Universitaria del Huila – CORHUILA

Inteligencia Artificial, semestre 2026-2

José Miguel Llanos Mosquera

10 de noviembre de 2026

Tabla de contenido

Introducción 5
Planteamiento y descripción del problema 5
Justificación 6
Objetivos 7
Objetivo general 7
Objetivos específicos 7
Marco teórico 8
Las emociones y su reconocimiento automático 8
Procesamiento de lenguaje natural y clasificación de textos 8
Representación de palabras mediante embeddings 8
Redes neuronales recurrentes y LSTM 9
LSTM bidireccional 9
Regularización y optimización 10
Métricas de evaluación y calibración 10
Metodología 10
Descripción y análisis de la base de datos 11
Origen y estructura 11
Análisis exploratorio 12
Preparación de los datos 15
División sin fuga de información 16
Tokenización y longitud de secuencia 17
Tratamiento del desbalance 17
Diseño y construcción de la red neuronal 18
Entrenamiento y configuración experimental 19
Evaluación y métricas 22
Análisis de errores y calibración 24
Desarrollo de la aplicación web 26
Despliegue en producción 30
Resultados obtenidos 32
Limitaciones y posibles mejoras 33
Conclusiones 34
Referencias 36
Anexos 40

Anexo A. Estructura del proyecto 40
Anexo B. Ejecución reproducible 40

Índice de tablas

Tabla 1 *Fases de CRISP-DM aplicadas al proyecto* 11
Tabla 2 *Estructura del archivo text.csv* 12
Tabla 3 *Distribución de textos por emoción antes y después de la limpieza* 12
Tabla 4 *Pasos de limpieza y filas afectadas* 15
Tabla 5 *División del conjunto limpio por emoción* 16
Tabla 6 *Arquitectura de la red neuronal* 18
Tabla 7 *Configuración del entrenamiento* 19
Tabla 8 *Historial de entrenamiento por época* 20
Tabla 9 *Métricas por emoción en el conjunto de prueba* 22
Tabla 10 *Predicciones del modelo en frases escritas para casos difíciles* 26
Tabla 11 *Rutas de la aplicación web* 26
Tabla 12 *Comparación de los motores de inferencia* 31

Índice de figuras

Figura 1 *Distribución de textos por emoción antes y después de la limpieza* 13
Figura 2 *Parejas de emociones con etiquetas contradictorias en el corpus original* 14
Figura 3 *Palabras más características de cada emoción en el conjunto de entrenamiento* 15
Figura 4 *Longitud de los textos de entrenamiento y longitud máxima del modelo* 17
Figura 5 *Evolución de la pérdida de entrenamiento y validación* 21
Figura 6 *Evolución de la exactitud de entrenamiento y validación* 21
Figura 7 *F1 por emoción de la red neuronal frente a la línea base* 23
Figura 8 *Matriz de confusión en el conjunto de prueba* 24
Figura 9 *Distribución de la confianza del modelo en aciertos y errores* 25
Figura 10 *Página de inicio con el formulario de análisis* 28
Figura 11 *Resultado de una predicción real con las probabilidades por emoción* 29
Figura 12 *Página de resultados del modelo* 30

Clasificación automática de emociones en textos en inglés mediante una red neuronal LSTM bidireccional integrada en una aplicación web

Introducción

Las personas expresan a diario cómo se sienten en redes sociales, reseñas, encuestas y mensajes de atención al cliente. Ese volumen de texto contiene información valiosa sobre el estado emocional de usuarios, clientes o estudiantes, pero su lectura manual resulta inviable cuando se cuentan miles o millones de mensajes. El Procesamiento de Lenguaje Natural (PLN) ofrece técnicas para automatizar esa lectura y, en particular, las redes neuronales permiten aprender directamente de ejemplos etiquetados qué patrones lingüísticos se asocian a cada emoción (Goodfellow et al., 2016).

Este informe documenta el desarrollo completo de un sistema que clasifica frases en inglés en seis emociones —tristeza, alegría, amor, ira, miedo y sorpresa—. El trabajo parte del conjunto de datos *Emotions* (Elgiriyewithana, 2024), compuesto por 416.809 mensajes, y sigue la metodología CRISP-DM (Wirth y Hipp, 2000): comprensión del problema y de los datos, preparación, modelado con una red neuronal LSTM bidireccional, evaluación sobre un conjunto de prueba independiente e implementación en una aplicación web desarrollada con Flask y preparada para su despliegue en la plataforma Render.

Todas las cifras, tablas y figuras de este documento proceden de la ejecución real del código del proyecto, incluida la verificación de la aplicación publicada en producción.

Planteamiento y descripción del problema

El problema que se aborda es una tarea de **clasificación de texto multiclase**: dada una frase escrita en inglés en la que una persona describe cómo se siente, el sistema debe determinar cuál de seis emociones predomina y con qué probabilidad. Se trata de un problema de PLN, no

de visión por computadora, porque los datos disponibles son exclusivamente textos con una etiqueta entera asociada.

La dificultad del problema no reside solo en el vocabulario. Una misma palabra puede aparecer en contextos emocionales distintos, las frases pueden mezclar varias emociones y el conjunto de datos presenta problemas de calidad que condicionan el aprendizaje: clases desbalanceadas, textos duplicados y, sobre todo, frases idénticas etiquetadas con emociones diferentes. Un modelo útil debe, además, poder utilizarse fuera del entorno de entrenamiento, por lo que el problema incluye su integración en una aplicación web que procese la entrada del usuario con las mismas transformaciones empleadas al entrenar.

La pregunta que guía el proyecto es la siguiente: ¿es posible construir, con recursos de cómputo modestos, una red neuronal que identifique la emoción predominante en frases en inglés con un rendimiento alto y equilibrado entre clases, y ponerla a disposición de usuarios no técnicos mediante una aplicación web?

Justificación

El reconocimiento automático de emociones tiene aplicaciones directas en la clasificación de comentarios de clientes, el análisis de opiniones en redes sociales, la investigación en ciencias sociales y la priorización de mensajes en servicios de atención. Disponer de un clasificador que entregue no solo una etiqueta sino la distribución de probabilidades permite que una persona revise los casos dudosos en lugar de leer todos los mensajes.

Desde el punto de vista académico, el proyecto integra los componentes principales de un sistema de inteligencia artificial en producción: análisis y limpieza de datos reales con defectos, diseño y entrenamiento de una red neuronal, evaluación rigurosa sin fuga de información, comparación con una línea base, desarrollo de una interfaz y preparación del despliegue. La

elección de una LSTM bidireccional, en lugar de un modelo de tipo Transformer, se justifica por el tamaño del corpus, la brevedad de los textos y la necesidad de entrenar en una computadora personal sin GPU, como se explica en la sección de diseño.

Finalmente, el proyecto tiene una dimensión ética: un clasificador de emociones no es una herramienta de diagnóstico psicológico. Por ello, la aplicación informa de sus limitaciones y advierte cuando la entrada se aleja de los datos con los que fue entrenada.

Objetivos

Objetivo general

Desarrollar y evaluar una red neuronal LSTM bidireccional que clasifique frases en inglés en seis emociones, e integrarla en una aplicación web funcional preparada para su despliegue en producción.

Objetivos específicos

1. Analizar la estructura, la calidad y la distribución de clases del conjunto de datos *Emotions*, identificando duplicados, etiquetas contradictorias y desbalance.
2. Preparar los datos mediante limpieza, deduplicación, tokenización y una división estratificada en entrenamiento, validación y prueba que evite la fuga de información.
3. Diseñar, entrenar y regularizar una red neuronal con capas de *embedding* y LSTM bidireccional, guardando el mejor modelo según la validación.
4. Evaluar el modelo en un conjunto de prueba independiente con exactitud, precisión, exhaustividad, F1 y matriz de confusión, y compararlo con una línea base clásica.
5. Implementar una aplicación web con Flask que valide la entrada, ejecute predicciones reales y muestre las probabilidades, la explicación del resultado y las gráficas de evaluación.

6. Preparar el despliegue de la aplicación en una plataforma en la nube compatible con un plan gratuito y documentar su verificación.

Marco teórico

Las emociones y su reconocimiento automático

Ekman (1992) defendió la existencia de un conjunto reducido de emociones básicas, con expresiones y funciones diferenciadas, entre las que se encuentran la ira, el miedo, la tristeza, la alegría y la sorpresa. El conjunto de datos utilizado comparte esas cinco categorías e incorpora el amor. Saravia et al. (2018) presentaron un corpus de mensajes de Twitter en inglés con este mismo esquema de seis emociones, construido mediante supervisión distante; es decir, las etiquetas se obtuvieron de forma automática a partir de indicios presentes en los propios mensajes y no mediante anotación manual. Esta forma de etiquetado permite reunir cientos de miles de ejemplos, pero introduce ruido, un aspecto que se confirma en el análisis de los datos de este proyecto.

Procesamiento de lenguaje natural y clasificación de textos

La clasificación de textos asigna a cada documento una o varias categorías. Un enfoque clásico consiste en representar el texto como un vector de frecuencias ponderadas, como TF-IDF, y aplicar un clasificador lineal (Manning et al., 2008). Este enfoque ignora el orden de las palabras, pero es rápido y suele ser una línea base exigente. Las redes neuronales, en cambio, pueden aprender representaciones que tienen en cuenta la secuencia y el contexto.

Representación de palabras mediante embeddings

Un *embedding* asigna a cada palabra del vocabulario un vector denso de números reales que se ajusta durante el entrenamiento. Palabras que aparecen en contextos similares tienden a

obtener vectores cercanos, lo que permite generalizar entre sinónimos (Mikolov et al., 2013). En este proyecto la capa de *embedding* se aprende desde cero junto con el resto de la red.

Redes neuronales recurrentes y LSTM

Las redes neuronales recurrentes procesan una secuencia elemento a elemento y mantienen un estado oculto que resume lo leído. Su principal limitación es la dificultad para aprender dependencias largas, porque los gradientes tienden a desvanecerse. La red *Long Short-Term Memory* (LSTM) de Hochreiter y Schmidhuber (1997) introduce una celda de memoria regulada por compuertas que deciden qué información se incorpora, se conserva o se expone. En la formulación implementada por Keras, para cada palabra de entrada x_t se calcula:

$$i_t = \sigma(W_i\, x_t + U_i\, h_{t-1} + b_i) \qquad f_t = \sigma(W_f\, x_t + U_f\, h_{t-1} + b_f)$$

$$g_t = \tanh(W_c\, x_t + U_c\, h_{t-1} + b_c) \qquad o_t = \sigma(W_o\, x_t + U_o\, h_{t-1} + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \qquad h_t = o_t \odot \tanh(c_t)$$

donde σ es la función sigmoide, $\odot$ el producto elemento a elemento, i_t, f_t y o_t las compuertas de entrada, olvido y salida, g_t el candidato a nuevo contenido, c_t el estado de la celda y h_t el estado oculto. Estas mismas ecuaciones se implementaron en NumPy para el despliegue, como se describe más adelante.

LSTM bidireccional

Una red recurrente bidireccional procesa la secuencia en ambos sentidos con dos redes independientes y combina sus estados (Schuster y Paliwal, 1997). Graves y Schmidhuber (2005) mostraron las ventajas de combinar esta idea con celdas LSTM. Para la clasificación de emociones resulta útil porque la interpretación de una palabra puede depender tanto de lo que la precede como de lo que la sigue.

Regularización y optimización

El *dropout* desactiva aleatoriamente una fracción de las neuronas durante el entrenamiento, lo que reduce la coadaptación y el sobreajuste (Srivastava et al., 2014). La parada temprana detiene el entrenamiento cuando la pérdida de validación deja de mejorar y conserva los pesos de la mejor época (Goodfellow et al., 2016). Los parámetros se optimizaron con Adam, un método de descenso de gradiente estocástico con tasas de aprendizaje adaptativas por parámetro (Kingma y Ba, 2015).

Métricas de evaluación y calibración

La exactitud mide la proporción de aciertos, pero puede ser engañosa con clases desbalanceadas. Por ello se emplean también la precisión, la exhaustividad (*recall*) y su media armónica, el F1, calculadas por clase y promediadas de forma macro, de modo que cada emoción pese lo mismo (Sokolova y Lapalme, 2009). La matriz de confusión muestra qué emociones se confunden entre sí. Además, Guo et al. (2017) observaron que las redes neuronales modernas suelen estar mal calibradas: su probabilidad declarada no coincide con su tasa real de acierto. Para medirlo se usa el error de calibración esperado (ECE).

Metodología

El proyecto se desarrolló siguiendo CRISP-DM (*Cross-Industry Standard Process for Data Mining*), un modelo de proceso de seis fases iterativas para proyectos de datos (Wirth y Hipp, 2000). La Tabla 1 relaciona cada fase con el trabajo realizado y con los artefactos del proyecto que la evidencian. A esas fases se añadió una etapa de seguimiento y mejoras, que se desarrolla en la sección de limitaciones.

Tabla 1

Fases de CRISP-DM aplicadas al proyecto

Fase	Trabajo realizado	Evidencia en el proyecto
Comprensión del problema	Definición de la tarea, usuarios y utilidad	Este informe
Comprensión de los datos	Estructura, calidad, distribución y vocabulario	src/eda.py, figuras de EDA
Preparación de los datos	Limpieza, deduplicación, división y tokenización	src/data_prep.py, src/tokenizer.py
Modelado	Red BiLSTM, línea base TF-IDF	src/model.py, src/train.py, src/baseline.py
Evaluación	Métricas en prueba, errores y calibración	src/evaluate.py, reports/metrics.json
Implementación	Aplicación Flask y despliegue en Render	app/, wsgi.py, render.yaml

Nota. Elaboración propia.

El entorno de trabajo fue Visual Studio Code con Python 3.13 en Windows 11, sin GPU. Se emplearon pandas para la manipulación de datos (McKinney, 2010), scikit-learn para la división de datos, la línea base y las métricas (Pedregosa et al., 2011), TensorFlow y Keras para la red neuronal (Abadi et al., 2016; Chollet et al., 2015), NumPy para la inferencia en producción (Harris et al., 2020), Matplotlib para las gráficas (Hunter, 2007) y Flask para la aplicación web (Pallets, s. f.). Todo el código es modular y reproducible: cada fase se ejecuta con un único comando y escribe sus resultados en la carpeta *reports*.

Descripción y análisis de la base de datos

Origen y estructura

Los datos provienen del conjunto *Emotions* publicado en Kaggle por Elgiriyewithana (2024), que reúne mensajes de Twitter en inglés etiquetados con la emoción que predomina en cada uno. El archivo recibido, *text.csv*, ocupa 44.916.111 bytes, la misma cifra que declara la ficha del conjunto en Kaggle, y su correspondencia de etiquetas es la documentada por el autor. La Tabla 2 resume su estructura.

Tabla 2

Estructura del archivo text.csv

Columna	Tipo	Descripción
Unnamed: 0	Entero	Índice consecutivo 0…n−1 guardado por pandas; no aporta información y se descarta
text	Texto	Mensaje en inglés, en minúsculas, sin signos de puntuación ni números
label	Entero 0–5	0 = tristeza, 1 = alegría, 2 = amor, 3 = ira, 4 = miedo, 5 = sorpresa

Nota. El archivo contiene 416.809 filas y ningún valor nulo.

Análisis exploratorio

Los textos son breves: tienen una media de 19,1 palabras y una mediana de 17, y el 99 % no supera las 52 palabras (máximo 178). El 98,7 % contiene las palabras *feel* o *feeling* y el 98,6 % empieza con «i» o «im», lo que indica que el corpus se compone casi por completo de frases en primera persona del tipo «I feel…». El texto ya venía normalizado, pero 5.734 filas conservaban restos de etiquetas HTML y direcciones web («a href http www»), y la palabra «amp» aparecía como residuo de la entidad HTML &.

La distribución de clases es desbalanceada (véase la Tabla 3 y la Figura 1). En el archivo original, alegría y tristeza suman el 62,9 % de los textos, mientras que sorpresa representa solo el 3,6 %. La relación entre la clase más frecuente y la menos frecuente es de 9,4 a 1.

Tabla 3

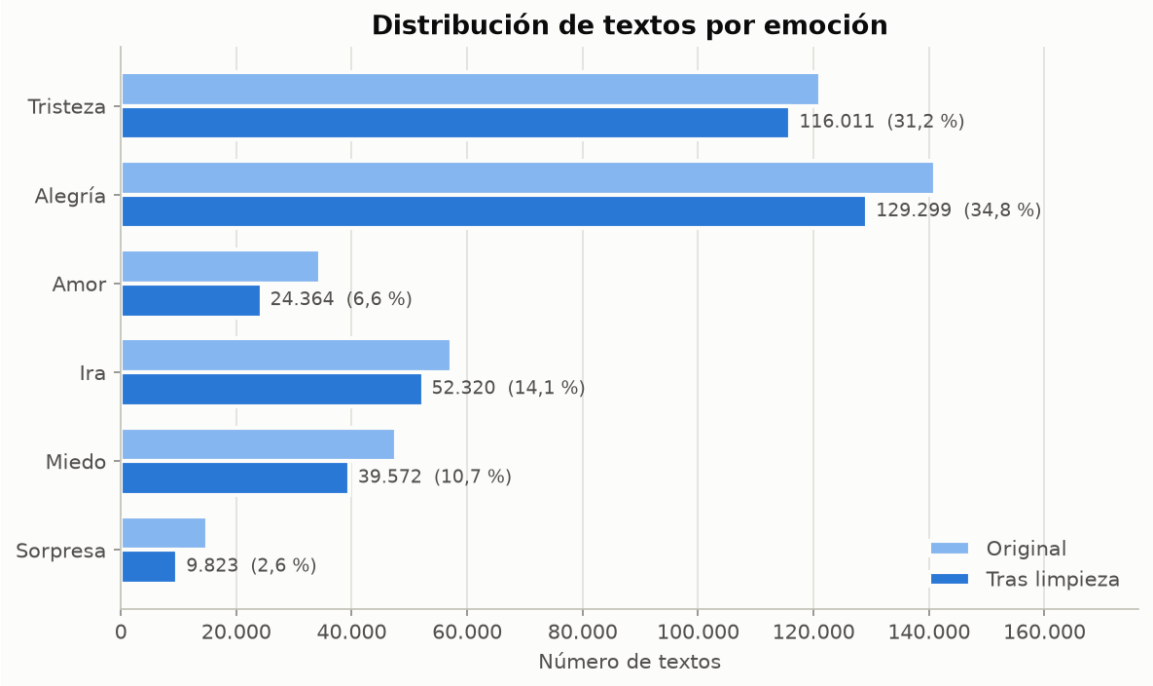
Distribución de textos por emoción antes y después de la limpieza

Emoción	Original	%	Tras limpieza	%	Variación
Tristeza	121.187	29,1 %	116.011	31,2 %	-4,3 %
Alegría	141.067	33,8 %	129.299	34,8 %	-8,3 %
Amor	34.554	8,3 %	24.364	6,6 %	-29,5 %
Ira	57.317	13,8 %	52.320	14,1 %	-8,7 %
Miedo	47.712	11,4 %	39.572	10,7 %	-17,1 %
Sorpresa	14.972	3,6 %	9.823	2,6 %	-34,4 %
Total	416.809	100 %	371.389	100 %	-10,9 %

Nota. La limpieza se describe en la sección siguiente.

Figura 1

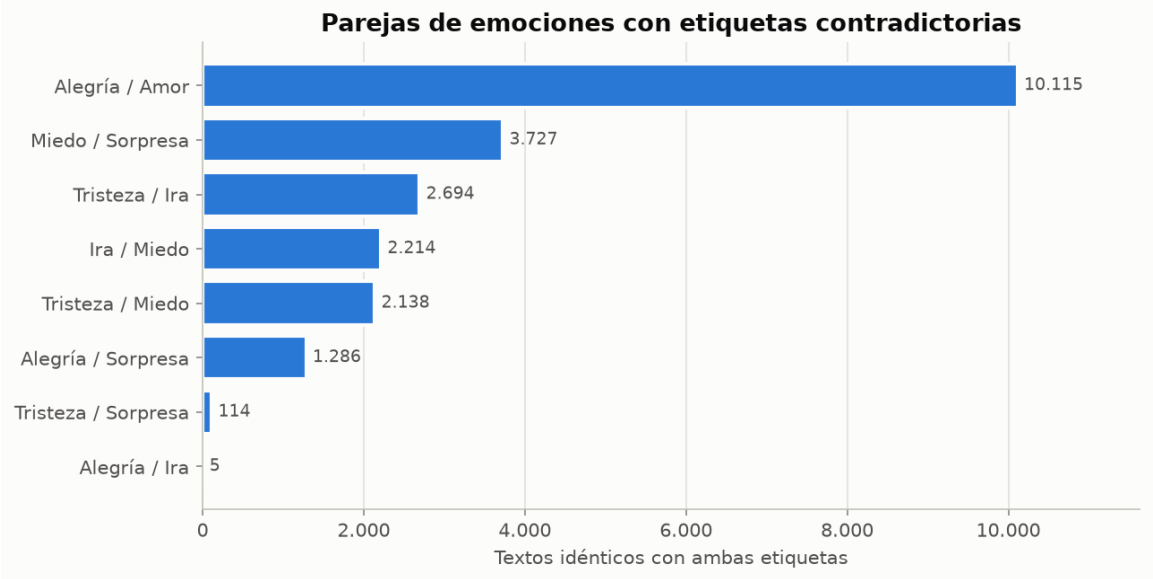
Distribución de textos por emoción antes y después de la limpieza



El hallazgo de calidad más relevante es la existencia de **22.288 textos idénticos etiquetados con emociones distintas**, que suman 44.582 filas (10,7 % del total). Los conflictos no son aleatorios: se concentran en emociones cercanas, como muestra la Figura 2. Las parejas más frecuentes son alegría–amor (10.115), miedo–sorpresa (3.727), tristeza–ira (2.694), ira–miedo (2.214), tristeza–miedo (2.138). Este patrón es coherente con un etiquetado automático y establece un límite a la exactitud alcanzable, porque para esos textos no existe una única respuesta correcta.

Figura 2

Parejas de emociones con etiquetas contradictorias en el corpus original

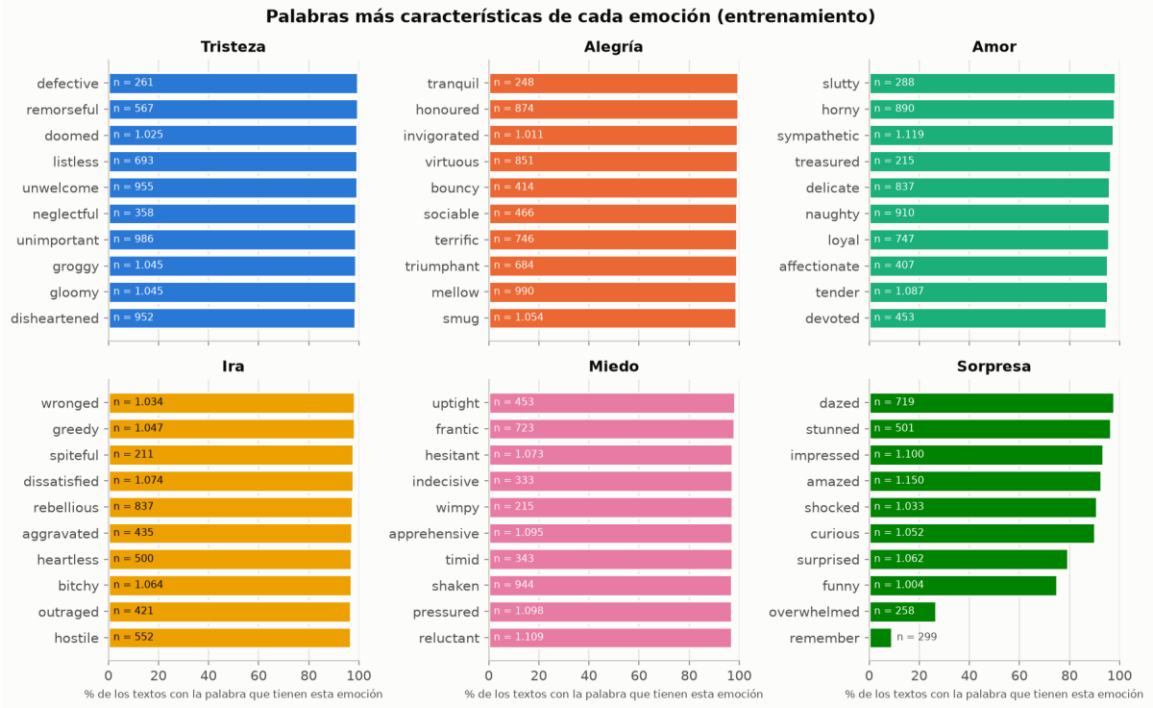


Nota. Cada barra cuenta los textos que aparecen con ambas etiquetas. Estos textos se excluyeron del conjunto limpio.

Para entender qué información usará el modelo, se calculó para cada palabra el porcentaje de los textos que la contienen y pertenecen a cada emoción, usando solo el conjunto de entrenamiento (Figura 3). Cada emoción tiene adjetivos casi exclusivos: *gloomy* o *listless* para tristeza, *apprehensive* o *hesitant* para miedo, *dazed* o *stunned* para sorpresa, entre otros, con porcentajes entre el 95 % y el 99 %. Sin embargo, solo 14 palabras superan el 99 % de exclusividad, y aparecen en apenas el 2,9 % de los textos. El vocabulario emocional es, por tanto, muy informativo, pero no determina la etiqueta de forma perfecta. La clase sorpresa es la menos «pura»: *surprised* pertenece a ella en el 79 % de los casos y *funny*, en el 75 %.

Figura 3

Palabras más características de cada emoción en el conjunto de entrenamiento



Nota. Se excluyeron las palabras vacías del inglés y las palabras con menos de 200 apariciones. n indica el número de textos de entrenamiento que contienen la palabra.

Preparación de los datos

La preparación se implementó en *src/data_prep.py* y *src/text_cleaning.py*. Cada decisión se tomó con base en el análisis anterior y se cuantificó, para que sus consecuencias fueran explícitas. La Tabla 4 resume los pasos.

Tabla 4

Pasos de limpieza y filas afectadas

Paso	Decisión y motivo	Filas
Normalización	Minúsculas, solo letras, eliminación de restos HTML y sustitución de «amp» por «and». Alinea la entrada de la aplicación con el formato del corpus	7.799 modificadas
Duplicados exactos	Mismo texto y misma etiqueta: no aportan información y podrían caer en entrenamiento y prueba a la vez	−838

Paso	Decisión y motivo	Filas
Etiquetas contradictorias	Mismo texto con varias etiquetas: no tienen una respuesta única; se excluyen para entrenar y evaluar con etiquetas fiables	−44.582
Textos vacíos tras limpiar	Se eliminarían; no se encontró ninguno	0
Resultado	Conjunto limpio con textos únicos	371.389

La exclusión de los textos contradictorios tiene un coste que conviene explicitar: afecta más a las clases minoritarias. Amor pierde 10.115 filas (29,3 % de su total) y sorpresa 5.127 (34,2 %). Aun así, ambas conservan 24.364 y 9.823 ejemplos, suficientes para el aprendizaje, y la evaluación se realiza sobre etiquetas sin ambigüedad.

División sin fuga de información

Como cada texto del conjunto limpio es único, una división estratificada por emoción garantiza que ninguna frase aparezca en dos subconjuntos. Se usaron proporciones de 80 %, 10 % y 10 % con una semilla fija (42). Una comprobación automática detiene el proceso si detecta textos compartidos entre subconjuntos; el resultado fue cero coincidencias en los tres pares. La Tabla 5 muestra que la proporción de cada emoción se mantiene en los tres subconjuntos.

Tabla 5

División del conjunto limpio por emoción

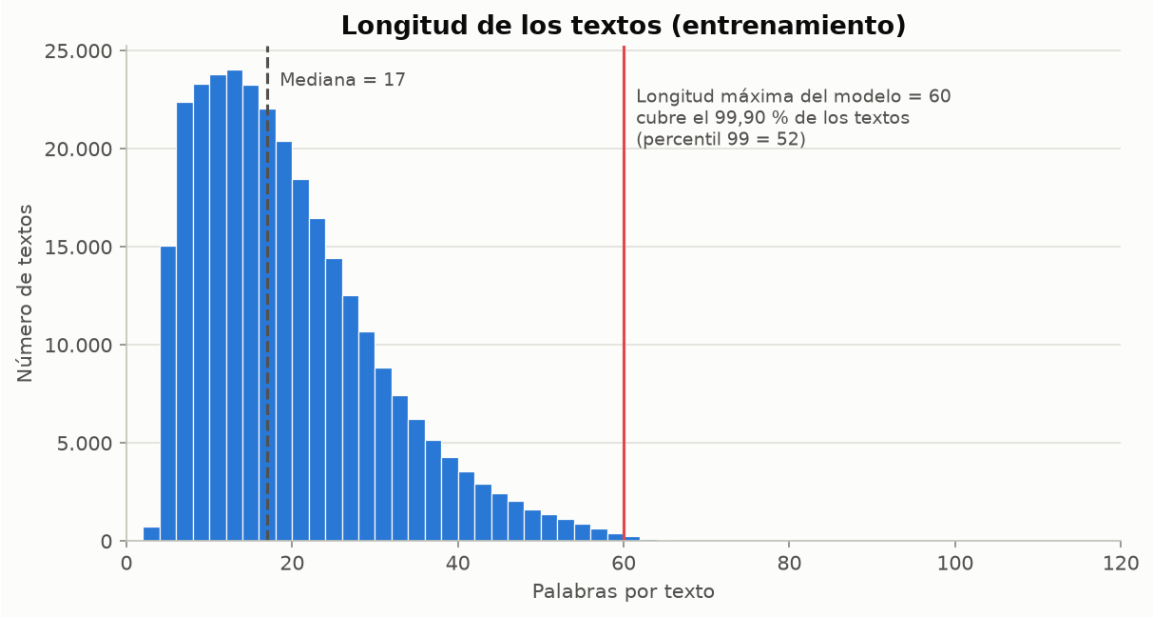
Emoción	Entrenamiento	Validación	Prueba
Tristeza	92.809	11.601	11.601
Alegría	103.439	12.930	12.930
Amor	19.490	2.437	2.437
Ira	41.856	5.232	5.232
Miedo	31.658	3.957	3.957
Sorpresa	7.859	982	982
Total	297.111	37.139	37.139

Tokenización y longitud de secuencia

El vocabulario se construyó **solo con el conjunto de entrenamiento**, para no filtrar información de validación o prueba. Se conservaron las 29.998 palabras más frecuentes, más dos índices reservados: 0 para el relleno y 1 para las palabras desconocidas. Con este vocabulario, solo el 0,95 % de las palabras de validación resultan desconocidas. Cada texto se convierte en una secuencia de 60 índices: se conservan las primeras 60 palabras y se rellena con ceros al final. Esta longitud cubre el 99,90 % de los textos de entrenamiento (Figura 4). El tokenizador se implementó en Python puro y se guarda en *models/tokenizer.json*, de modo que la aplicación web aplica exactamente la misma transformación.

Figura 4

Longitud de los textos de entrenamiento y longitud máxima del modelo



Tratamiento del desbalance

En lugar de eliminar ejemplos de las clases mayoritarias o duplicar los de las minoritarias, se ponderó la función de pérdida con pesos inversamente proporcionales a la

frecuencia de cada clase, calculados con la fórmula $n / (k \cdot n_c)$, donde n es el número de textos, k el número de clases y n_c los textos de la clase c. Los pesos resultantes fueron: sorpresa 6,30, amor 2,54, miedo 1,56, ira 1,18, tristeza 0,53 y alegría 0,48. Así, un error en un texto de sorpresa penaliza al modelo unas 13 veces más que uno en un texto de alegría.

Diseño y construcción de la red neuronal

La arquitectura, definida en *src/model.py* con Keras, se resume en la Tabla 6. La capa de *embedding* convierte cada índice en un vector de 128 dimensiones; *mask_zero* indica a la LSTM que ignore el relleno. *SpatialDropout1D* apaga dimensiones completas del *embedding* durante el entrenamiento. La capa LSTM bidireccional, con 64 unidades por sentido, lee la frase en ambos sentidos y devuelve un vector de 128 valores que resume su contenido. A continuación, una capa densa con activación ReLU combina esas características y una capa de salida *softmax* produce seis probabilidades que suman 1.

Tabla 6

Arquitectura de la red neuronal

Capa	Configuración	Salida	Parámetros
Entrada	60 índices de palabra	(60)	0
Embedding	30.000 × 128, mask_zero = True	(60, 128)	3.840.000
SpatialDropout1D	tasa 0,2	(60, 128)	0
Bidirectional(LSTM)	64 unidades por sentido	(128)	98.816
Dropout	tasa 0,3	(128)	0
Dense	64 unidades, ReLU	(64)	8.256
Dropout	tasa 0,3	(64)	0
Dense (salida)	6 unidades, softmax	(6)	390
Total			3.947.462

Nota. Parámetros entrenables: 3.947.462 (unos 15 MB en coma flotante de 32 bits).

La función de pérdida es la entropía cruzada categórica en su versión *sparse*, apropiada cuando las etiquetas son enteros, y el optimizador es Adam con una tasa de aprendizaje inicial de 0,001. La métrica monitorizada durante el entrenamiento es la exactitud.

La arquitectura se eligió por su adecuación al problema y a los recursos disponibles, no por su complejidad. Los textos son cortos y el corpus es grande, condiciones en las que una red recurrente bidireccional aprende bien. Un modelo de tipo Transformer preentrenado, como BERT, podría captar mejor fenómenos como la negación, pero ocupa cientos de megabytes, requiere GPU para entrenarse en un tiempo razonable y no cabría en un servidor gratuito de 512 MB. La red elegida tiene menos de cuatro millones de parámetros y se entrenó en menos de veinte minutos en una CPU.

Entrenamiento y configuración experimental

El entrenamiento se ejecutó con *src/train.py* en una CPU, sin GPU, con la configuración de la Tabla 7. Se fijó la semilla aleatoria para favorecer la reproducibilidad. Tres mecanismos controlaron el proceso. *ModelCheckpoint* guardó el modelo con menor pérdida de validación. *EarlyStopping* detuvo el entrenamiento tras tres épocas sin mejora y restauró los mejores pesos. *ReduceLROnPlateau* redujo a la mitad la tasa de aprendizaje cuando la pérdida de validación se estancaba. El historial completo se registró en *reports/training_history.csv*.

Tabla 7

Configuración del entrenamiento

Parámetro	**Valor**
Optimizador	Adam, tasa de aprendizaje inicial 0,001
Función de pérdida	Entropía cruzada categórica (sparse), ponderada por clase
Tamaño de lote	256
Épocas máximas / ejecutadas	12 / 6
Parada temprana	paciencia 3 sobre la pérdida de validación, restaurando los mejores pesos
Reducción de la tasa	factor 0,5 con paciencia 1 (mínimo 0,00001)

Parámetro	Valor
Semilla aleatoria	42
Hardware y tiempo	CPU, 19,8 minutos en total
Software	Python 3.13, TensorFlow 2.21.0, Keras 3.15.1

La Tabla 8, la Figura 5 y la Figura 6 muestran la evolución del aprendizaje. Ya en la primera época la exactitud de validación alcanzó el 98,37 %. La mejor pérdida de validación (0,0262) se obtuvo en la época 3, con una exactitud de validación del 99,14 %. Desde entonces, la pérdida de entrenamiento siguió bajando mientras la de validación aumentaba ligeramente, que es la señal típica del inicio del sobreajuste. La parada temprana detuvo el proceso en la época 6 y se conservó el modelo de la época 3.

Tabla 8

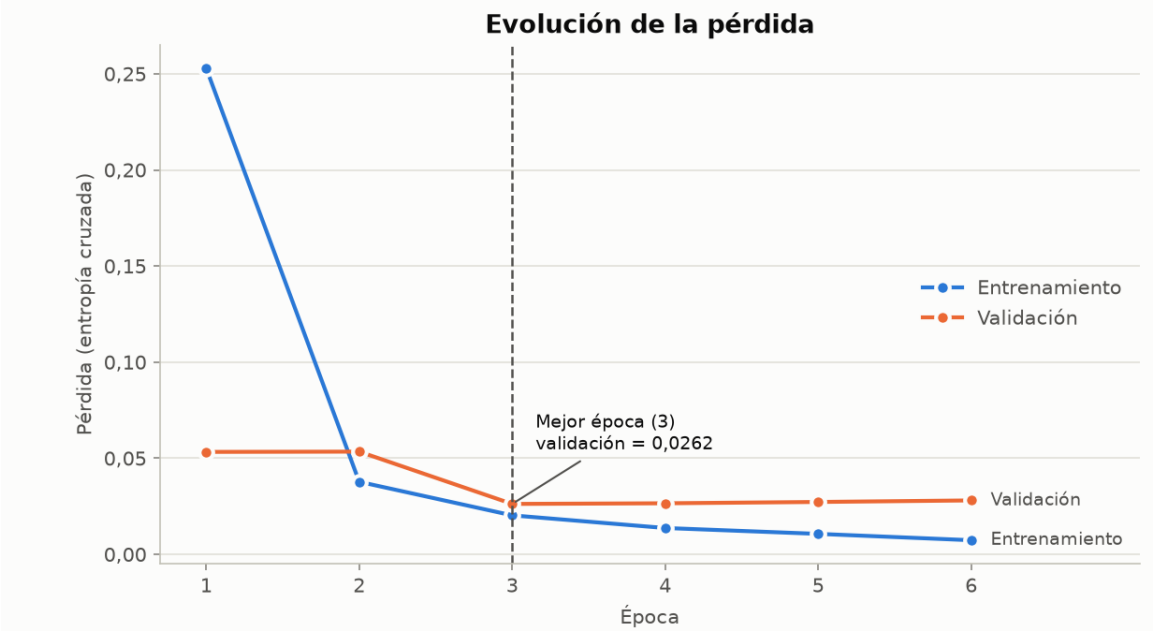
Historial de entrenamiento por época

Época	Pérdida (entr.)	Exactitud (entr.)	Pérdida (val.)	Exactitud (val.)	Tasa de aprendizaje	Segundos
1	0,2529	90,54 %	0,0533	98,37 %	0,0010	205
2	0,0377	98,77 %	0,0534	98,24 %	0,0010	218
3 (mejor)	0,0203	99,27 %	0,0262	99,14 %	5,00e-4	197
4	0,0137	99,50 %	0,0266	99,24 %	5,00e-4	192
5	0,0106	99,62 %	0,0272	99,21 %	2,50e-4	191
6	0,0073	99,73 %	0,0280	99,23 %	1,25e-4	183

Nota. La pérdida de entrenamiento está ponderada por clase y se calcula con *dropout* activo; la de validación no, por lo que no son directamente comparables en valor absoluto.

Figura 5

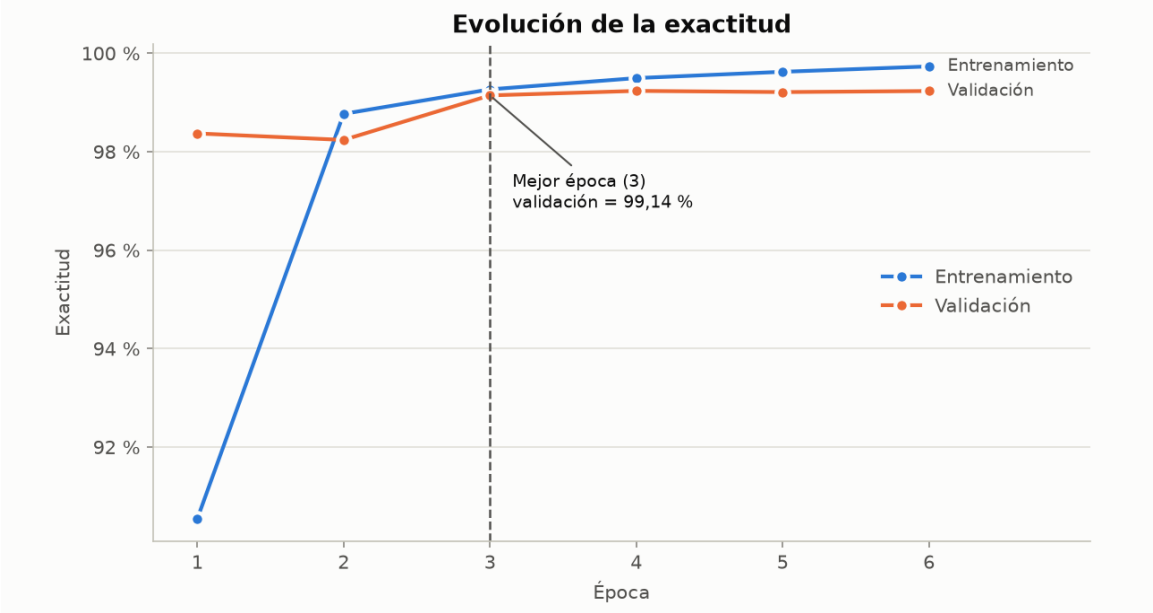
Evolución de la pérdida de entrenamiento y validación



Nota. La pérdida de entrenamiento está ponderada por clase y se calcula con dropout activo; la de validación no.

Figura 6

Evolución de la exactitud de entrenamiento y validación



Nota. La exactitud de entrenamiento se promedia durante la época con dropout activo, por eso puede quedar por debajo de la de validación al inicio.

Como referencia se entrenó una línea base clásica (*src/baseline.py*): un vector TF-IDF de unigramas y bigramas con 200.000 rasgos y una regresión logística con pesos de clase balanceados. Esta línea base se ajustó únicamente con el conjunto de entrenamiento y tardó 6,1 minutos. Su propósito es comprobar si la complejidad de la red neuronal aporta una mejora real.

Evaluación y métricas

La evaluación final se realizó con *src/evaluate.py* sobre los 37.139 textos del conjunto de prueba. Estos textos no se usaron para entrenar ni para elegir hiperparámetros. Para obtener las predicciones se usó el mismo flujo de la aplicación web: limpieza, tokenización y modelo guardado. La red alcanzó una **exactitud del 99,27 %**, con 272 errores, y un **F1 macro de 0,9898**. La línea base obtuvo una exactitud del 96,46 % y un F1 macro de 0,9543. La Tabla 9 detalla los resultados por emoción.

Tabla 9

Métricas por emoción en el conjunto de prueba

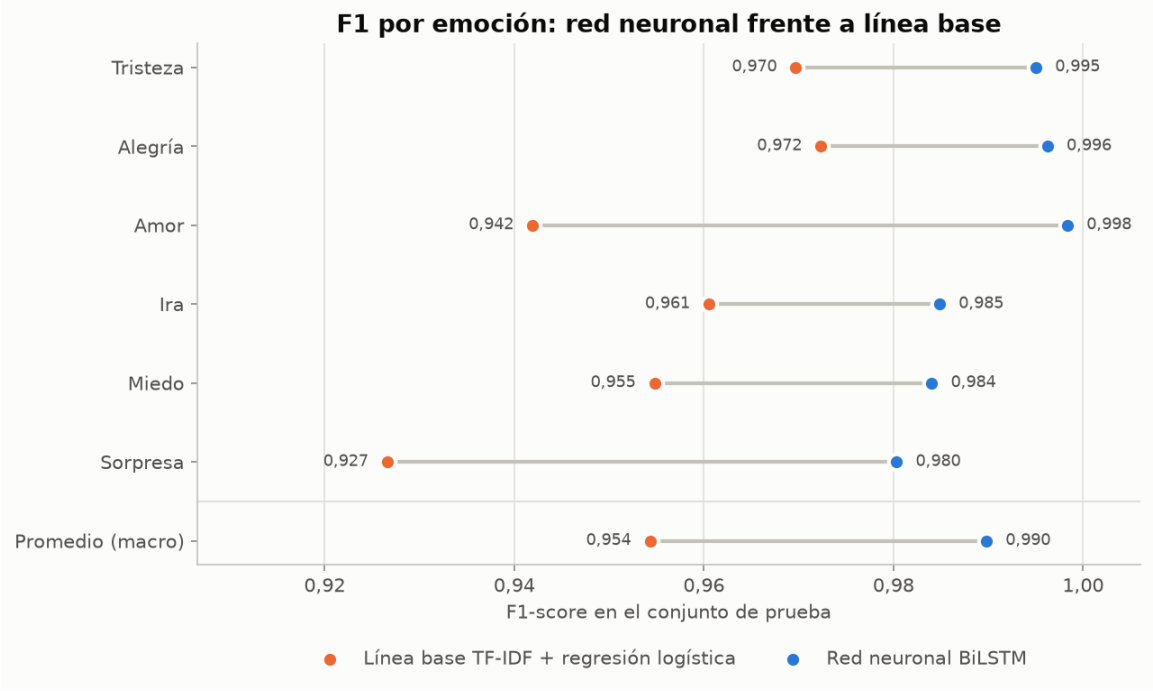
Emoción	Precisión	Exhaustividad	F1	F1 línea base	Textos
Tristeza	0,996	0,994	0,995	0,970	11.601
Alegría	0,997	0,996	0,996	0,972	12.930
Amor	0,998	0,999	0,998	0,942	2.437
Ira	0,981	0,989	0,985	0,961	5.232
Miedo	0,986	0,982	0,984	0,955	3.957
Sorpresa	0,971	0,990	0,980	0,927	982
Promedio macro	0,988	0,991	0,990	0,954	37.139
Promedio ponderado	0,993	0,993	0,993	0,965	37.139

Nota. La línea base es TF-IDF con regresión logística.

Todas las emociones superan un F1 de 0,98, a pesar de que la clase mayoritaria tiene trece veces más ejemplos que la minoritaria; la ponderación de la pérdida cumplió su función. La mejora frente a la línea base es mayor justamente en las clases minoritarias: amor pasa de 0,942 a 0,998 y sorpresa de 0,927 a 0,980 (Figura 7).

Figura 7

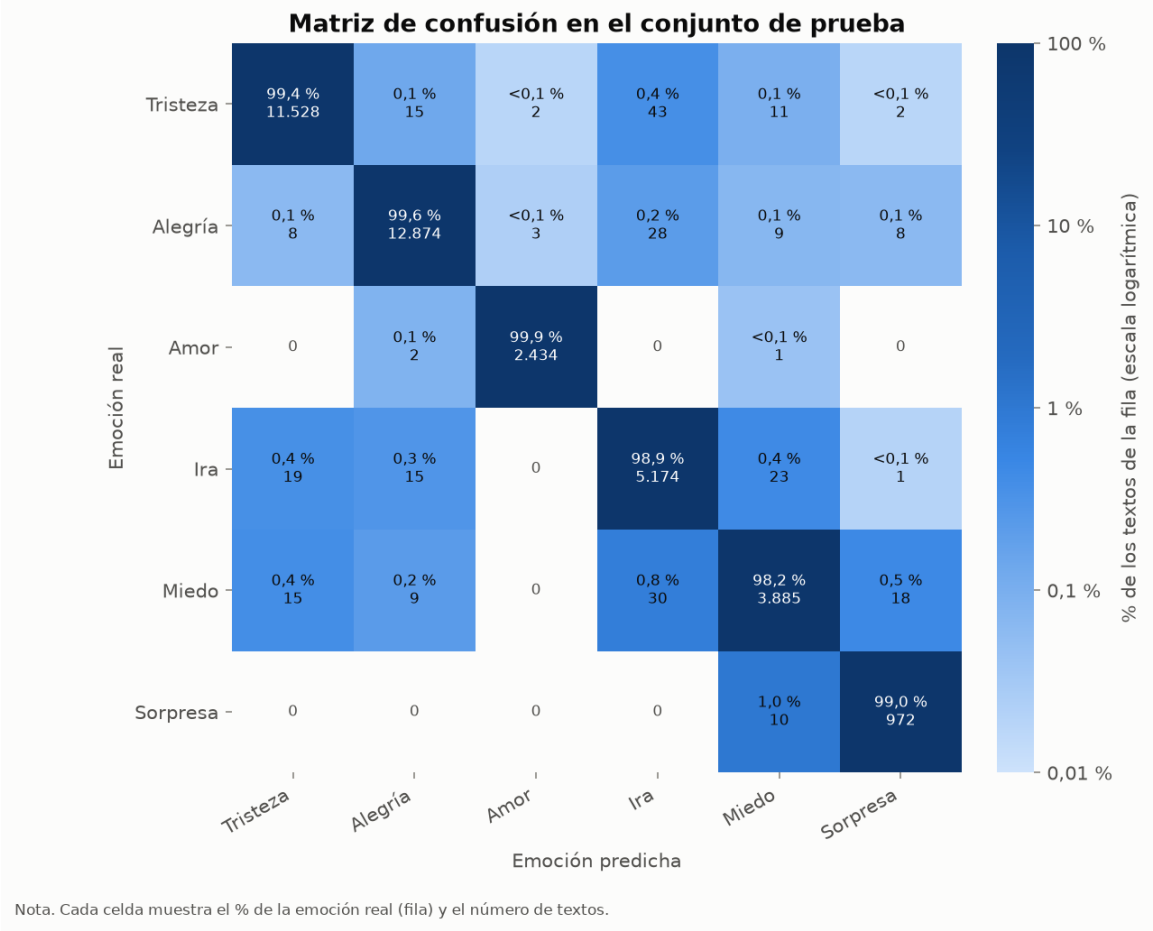
F1 por emoción de la red neuronal frente a la línea base



La matriz de confusión (Figura 8) muestra que los errores se concentran en pares de emociones negativas. Las confusiones más frecuentes son tristeza predicha como ira (43), miedo predicha como ira (30), alegría predicha como ira (28), ira predicha como miedo (23). La sorpresa solo se confunde con el miedo, lo que coincide con los conflictos de etiquetado observados en el análisis exploratorio.

Figura 8

Matriz de confusión en el conjunto de prueba



Nota. Cada celda indica el porcentaje de la emoción real (fila) y el número de textos. La escala de color es logarítmica para que los errores pequeños sean visibles.

Análisis de errores y calibración

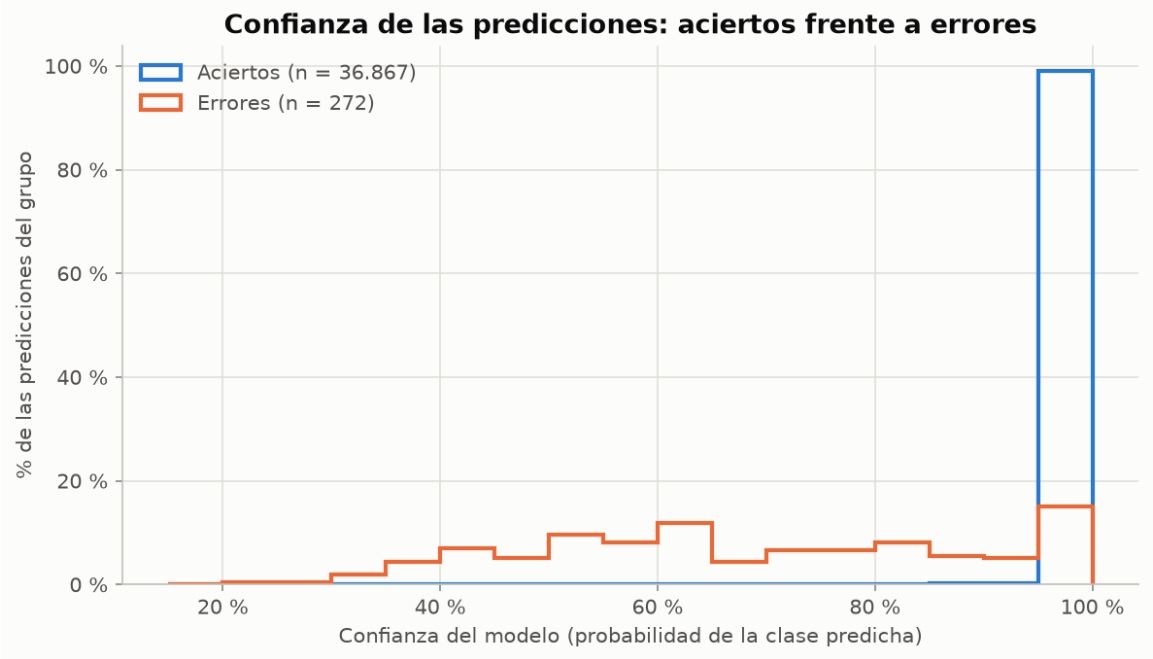
La revisión de los errores en que el modelo estaba más seguro (*reports/test_errors_sample.csv*) revela que una parte de ellos son, probablemente, errores de etiqueta del conjunto de datos. Por ejemplo, «i feel nervous anxious» está etiquetado como tristeza y el modelo predice miedo, y «i feel lousy and sleepy in the evening» está etiquetado como alegría y el modelo predice tristeza. En esos casos la predicción parece más razonable que

la etiqueta, lo que sugiere que la exactitud obtenida se acerca al límite que permite un etiquetado automático.

En cuanto a la confianza (Figura 9), la probabilidad media asignada a la clase predicha es del 99,7 % en los aciertos y del 68,8 % en los errores, por lo que una confianza baja es una buena señal de posible error. El error de calibración esperado es bajo (ECE = 0,0024), pero el 20,2 % de los errores se produjo con una confianza superior al 90 %. En consecuencia, una confianza alta no garantiza un acierto, en línea con lo señalado por Guo et al. (2017).

Figura 9

Distribución de la confianza del modelo en aciertos y errores



Para explorar el comportamiento fuera del corpus se escribieron cinco frases de prueba (Tabla 10). Sus resultados son reales, pero ilustrativos: no forman parte de las métricas. El modelo no maneja la negación, elige una sola emoción cuando la frase mezcla varias, asigna una emoción incluso a frases neutras y produce resultados sin sentido con textos en español, cuyas palabras casi no conoce.

Tabla 10

Predicciones del modelo en frases escritas para casos difíciles

Caso	Frase	Predicción	Confianza	Palabras conocidas
Negación	i am not happy at all today	alegría	86,4 %	7/7
Negación	i don t feel scared anymore	miedo	>99,9 %	6/6
Emociones mezcladas	i feel happy but also a little scared about the future	alegría	>99,9 %	11/11
Sin emoción explícita	the meeting is scheduled for monday at ten	alegría	41,2 %	8/8
Texto en español	me siento muy triste hoy	ira	86,2 %	1/5

Desarrollo de la aplicación web

La aplicación se desarrolló con Flask, un microframework de Python (Pallets, s. f.). Se prefirió a Django porque la aplicación necesita pocas rutas y ninguna base de datos, administración ni autenticación; Flask permite cubrir esas necesidades con poco código, fácil de mantener y de explicar. La Tabla 11 resume las rutas implementadas en *app/routes.py*.

Tabla 11

Rutas de la aplicación web

Ruta	Método	Función
/	GET, POST	Página de inicio con la explicación del problema, el formulario, el resultado y las limitaciones
/resultados	GET	Métricas reales, tabla por emoción, datos del entrenamiento y nueve gráficas
/figuras/<nombre>	GET	Sirve solo las figuras generadas por la evaluación (lista cerrada)
/api/predict	POST	Predicción en formato JSON para pruebas automatizadas
/health	GET	Comprueba que el modelo está cargado y predice; la usa el servidor de despliegue

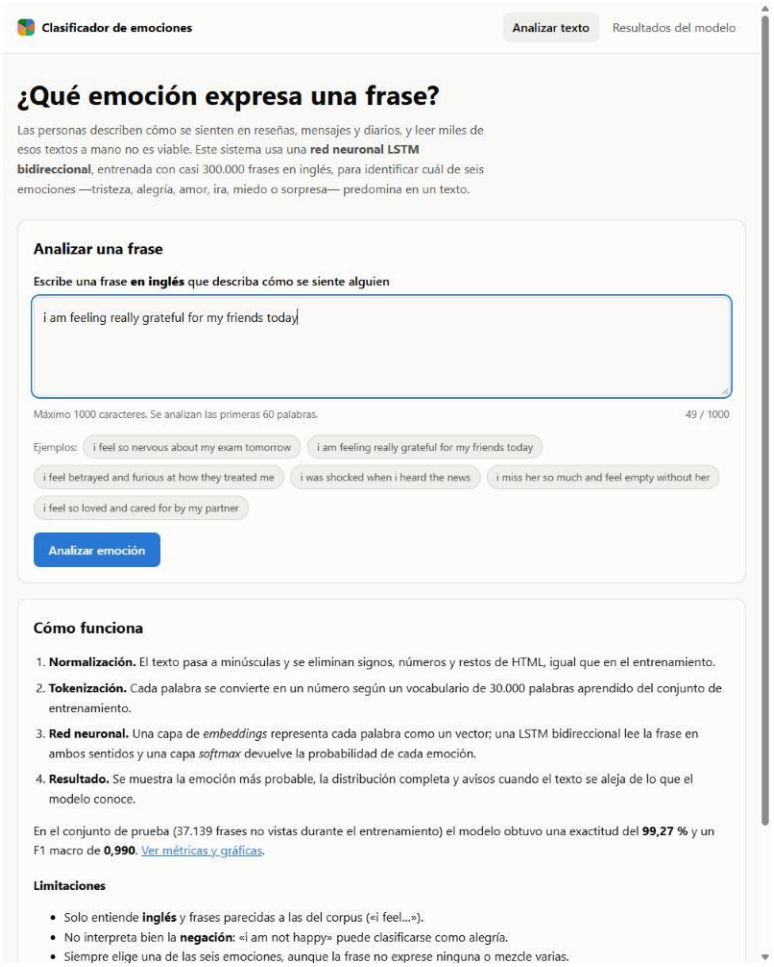
El flujo de una predicción es el siguiente. Primero se valida la entrada: se rechazan los textos vacíos, los de más de 1.000 caracteres y los que no contienen letras, con un mensaje claro y el código HTTP 400. Después, el texto pasa por la misma función de limpieza y el mismo

tokenizador del entrenamiento. A continuación, el modelo, cargado una sola vez al iniciar el servidor y nunca reentrenado durante una petición, calcula las seis probabilidades. Por último, la aplicación muestra la emoción más probable, una barra para cada emoción y una explicación en lenguaje natural.

Para que los resultados no induzcan a error, la aplicación emite advertencias en tres situaciones. La primera es cuando menos del 60 % de las palabras pertenece al vocabulario del modelo, como ocurre con textos en español; en ese caso el resultado se marca como «poco fiable». La segunda, cuando la confianza es inferior al 60 %. La tercera, cuando el texto supera las 60 palabras y se trunca. Las probabilidades extremas se muestran como «>99,9 %» o «<0,1 %», porque nunca son exactamente 1 o 0. La interfaz es adaptable: se verificó a 375 píxeles de ancho sin desplazamiento horizontal. La Figura 10, la Figura 11 y la Figura 12 muestran la aplicación en funcionamiento.

Figura 10

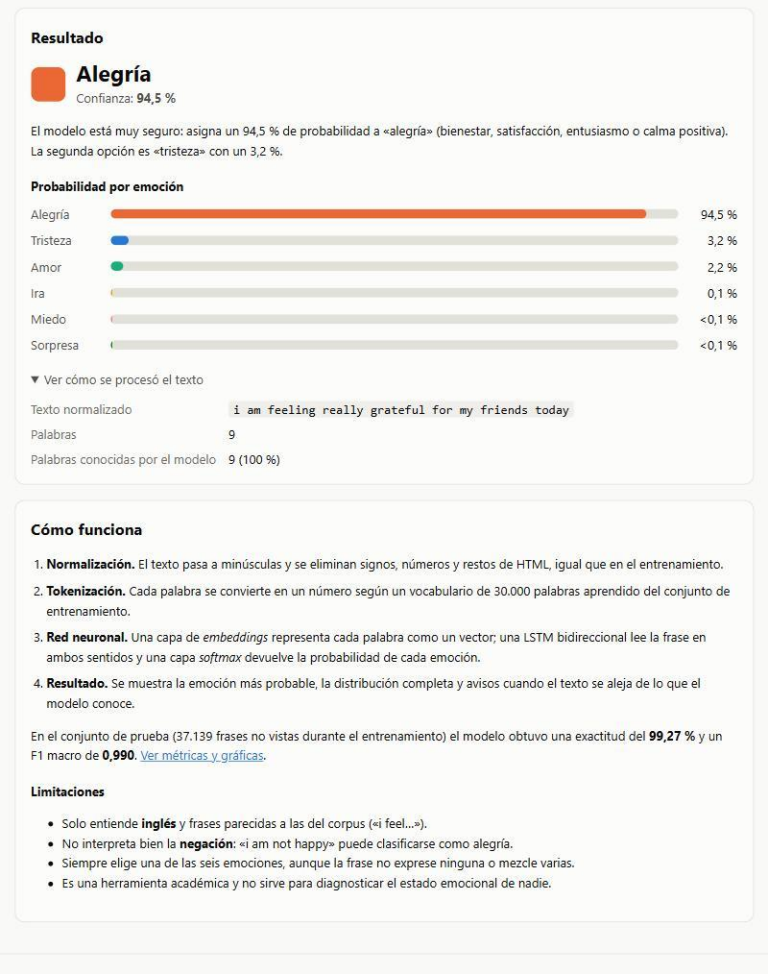
Página de inicio con el formulario de análisis



Nota. Captura de la aplicación ejecutándose en local.

Figura 11

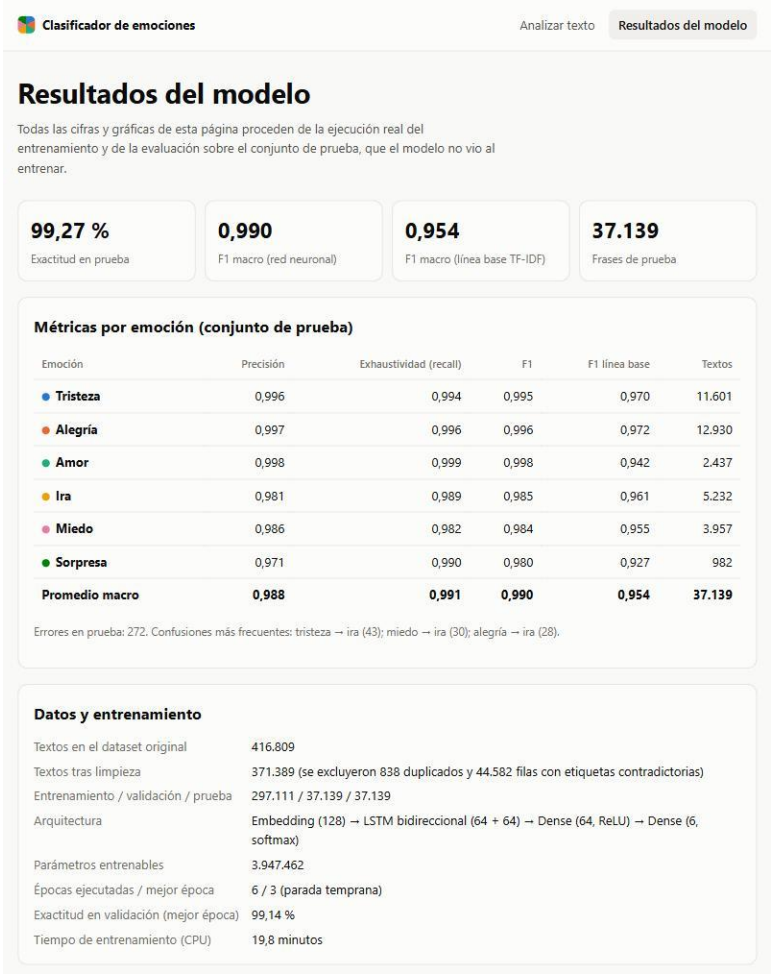
Resultado de una predicción real con las probabilidades por emoción



Nota. Frase analizada: «i am feeling really grateful for my friends today».

Figura 12

Página de resultados del modelo



Nota. La página continúa con las nueve gráficas de evaluación y de análisis de datos.

La aplicación cuenta con pruebas automatizadas con pytest. Cubren la validación de entradas, las predicciones por formulario y por API, que las probabilidades sumen 1, la advertencia con texto en español, las páginas de error, la ruta de salud y la página de resultados. En total, las 20 pruebas del proyecto se ejecutan con éxito.

Despliegue en producción

Se eligió Render como plataforma porque ofrece un plan gratuito para servicios web en Python y permite desplegar directamente desde un repositorio de GitHub. Según su

documentación, el plan gratuito dispone de 0,1 CPU y 512 MB de memoria (Render, s. f.-a). El servicio se suspende tras 15 minutos sin tráfico, tarda alrededor de un minuto en reactivarse y cuenta con 750 horas de instancia al mes (Render, s. f.-b).

Antes de desplegar se midió la memoria de la aplicación. Con TensorFlow cargado, el proceso ocupaba 431 MB de memoria física, demasiado cerca del límite de 512 MB. Para resolverlo se separó el entrenamiento de la inferencia: los pesos entrenados por Keras se exportaron a *models/emotion_bilstm_weights.npz* y la misma red se reimplementó con NumPy en *src/numpy_model.py*, siguiendo las ecuaciones de la LSTM presentadas en el marco teórico. La equivalencia se verificó en los 37.139 textos de validación (Tabla 12).

Tabla 12

Comparación de los motores de inferencia

Aspecto	**Keras/TensorFlow**	**NumPy**
Memoria física del proceso de la aplicación	431 MB	62 MB (pico de 72 MB)
Dependencias de producción	TensorFlow, Keras, h5py y otras	Flask, gunicorn y NumPy
Exactitud en validación	99,1438 %	99,1438 %
Diferencia máxima de probabilidad	—	0,0000027
Predicciones coincidentes	—	100 %
Tiempo por frase	—	1,4 ms

Nota. Mediciones realizadas en Windows 11. La memoria corresponde al conjunto de trabajo del proceso tras cargar el modelo y atender peticiones.

La configuración de despliegue está en el repositorio. El archivo *requirements.txt* fija las versiones de producción (Flask 3.1.3, gunicorn 26.2.0 y NumPy 2.5.4), mientras que las bibliotecas de entrenamiento quedan en *requirements-dev.txt*. El archivo *.python-version* fija Python 3.13; sin él, Render usaría por defecto una versión más reciente (Render, s. f.-c). El archivo *render.yaml* define el comando de construcción, el comando de inicio con gunicorn (un proceso, cuatro hilos y el puerto asignado por Render), la ruta de salud */health* y las variables de

entorno. El proyecto no utiliza contraseñas ni claves API. Los artefactos del modelo viajan en el repositorio porque ninguno supera los 16 MB.

La configuración de producción se verificó en local, en un entorno virtual limpio que solo contenía las dependencias de *requirements.txt*. La aplicación arrancó sin TensorFlow y la prueba de humo (*scripts/smoke_test.py*) superó sus siete comprobaciones: salud del modelo, tres predicciones, rechazo de un texto vacío, página de resultados y una figura. Como gunicorn no funciona en Windows, esa verificación usó el servidor de Flask.

Estado del despliegue: la aplicación está publicada en Render desde el 9 de noviembre de 2026 en https://clasificador-emociones-qmng.onrender.com, a partir del repositorio público https://github.com/soymiguelgomez/clasificador-emociones. La construcción usó Python 3.13.18, y la prueba de humo ejecutada contra la dirección pública superó sus siete comprobaciones. El modelo cargó con el motor NumPy y las tres predicciones de control fueron correctas (miedo, amor y tristeza). El texto vacío se rechazó con el código 400, y la página de resultados y sus figuras se sirvieron sin errores. El panel de Render no muestra el consumo de memoria en el plan gratuito, por lo que esa cifra procede de la medición local. Los pasos seguidos están documentados en *docs/DESPLIEGUE.md*.

Resultados obtenidos

Los resultados se contrastan con los objetivos específicos:

- **Análisis de datos.** Se caracterizó el conjunto completo y se identificaron 838 duplicados exactos, 22.288 textos con etiquetas contradictorias y un desbalance de 9,4 a 1.
- **Preparación.** Se obtuvo un conjunto limpio de 371.389 textos únicos, dividido en 297.111, 37.139 y 37.139 textos, sin ninguna coincidencia entre subconjuntos.

- **Modelado.** La red BiLSTM de 3.947.462 parámetros alcanzó su mejor validación en la época 3, con una exactitud del 99,14 %.
- **Evaluación.** En prueba obtuvo una exactitud del 99,27 % y un F1 macro de 0,990, frente a 0,954 de la línea base; todas las emociones superan un F1 de 0,98.
- **Aplicación web.** Se implementó en Flask con validación, predicción real, probabilidades, explicación, advertencias, gestión de errores y una página con nueve gráficas reales; las 20 pruebas automatizadas pasan.
- **Despliegue.** La aplicación está publicada en Render (https://clasificador-emociones-qmng.onrender.com) con una configuración de 62 MB que cabe en el plan gratuito, y superó en producción las siete comprobaciones de la prueba de humo.

Limitaciones y posibles mejoras

El modelo presenta limitaciones que deben tenerse en cuenta al interpretar sus resultados:

- **Idioma y dominio.** Solo procesa inglés y frases parecidas a las del corpus, en primera persona y con el verbo *feel*. Fuera de ese dominio su rendimiento no está medido.
- **Negación y composición.** No interpreta la negación («i am not happy» se clasifica como alegría) y elige una sola emoción cuando la frase mezcla varias.
- **Ausencia de clase neutra.** Siempre asigna una de las seis emociones, aunque el texto no exprese ninguna.
- **Ruido de etiquetado.** Las etiquetas son automáticas: se excluyó el 10,7 % de las filas por contradicciones y algunos errores restantes parecen fallos de etiqueta, lo que limita la validez de las métricas como medida de comprensión emocional.
- **Confianza.** Una probabilidad alta no garantiza un acierto: el 20 % de los errores tuvo una confianza superior al 90 %.

- **Uso responsable.** No es una herramienta de diagnóstico psicológico ni debe usarse para tomar decisiones sobre personas.

Como posibles mejoras se proponen las siguientes. Ajustar un modelo Transformer preentrenado multilingüe, que manejaría mejor la negación y el español, a cambio de un servidor con más memoria. Añadir ejemplos de negación y una clase neutra mediante anotación manual. Pasar a una formulación multietiqueta que aproveche los textos con varias emociones en lugar de excluirlos. Calibrar las probabilidades con escalado de temperatura (Guo et al., 2017). Y, en producción, registrar de forma anónima las entradas con advertencias para detectar cambios en el tipo de texto que reciben los usuarios.

Conclusiones

El proyecto demuestra que una red neuronal LSTM bidireccional de tamaño moderado, entrenada en una computadora personal sin GPU, puede clasificar la emoción predominante en frases en inglés con una exactitud del 99,27 % y un rendimiento equilibrado entre clases (F1 macro de 0,990). Además, supera con claridad a una línea base clásica, especialmente en las emociones con menos ejemplos.

El análisis de los datos fue tan determinante como el modelo. La detección de más de 22.000 textos con etiquetas contradictorias permitió entrenar y evaluar con etiquetas fiables y evitar la fuga de información. También explica por qué los errores restantes se concentran en emociones cercanas y por qué las métricas, aunque altas, deben interpretarse junto con las limitaciones del etiquetado.

En el plano de la ingeniería, separar el entrenamiento de la inferencia resultó clave para llevar el modelo a producción. Reimplementar la red en NumPy con los pesos entrenados redujo la memoria de 431 a 62 MB sin alterar ninguna predicción, lo que hace viable el despliegue en

un plan gratuito. Finalmente, la aplicación web acerca el modelo a usuarios no técnicos y muestra de forma explícita la incertidumbre y los límites del sistema, una condición necesaria para un uso responsable de la inteligencia artificial.

Referencias

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., … Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. En *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)* (pp. 265–283). USENIX Association. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

Chollet, F., et al. (2015). *Keras* [Software]. https://keras.io

Ekman, P. (1992). An argument for basic emotions. *Cognition and Emotion, 6*(3–4), 169–200. https://doi.org/10.1080/02699939208411068

Elgiriyewithana, N. (2024). *Emotions* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/nelgiriyewithana/emotions

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org

Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks, 18*(5–6), 602–610. https://doi.org/10.1016/j.neunet.2005.06.042

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. En *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1321–1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van

Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR 2015)*. https://doi.org/10.48550/arXiv.1412.6980

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space* (arXiv:1301.3781). arXiv. https://doi.org/10.48550/arXiv.1301.3781

Pallets. (s. f.). *Flask documentation* (versión 3.1.x). Recuperado el 29 de octubre de 2026, de https://flask.palletsprojects.com/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in

Python. *Journal of Machine Learning Research, 12*(85), 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Render. (s. f.-a). *Compute plans*. Recuperado el 29 de octubre de 2026, de https://render.com/docs/compute-plans

Render. (s. f.-b). *Deploy for free*. Recuperado el 29 de octubre de 2026, de https://render.com/docs/free

Render. (s. f.-c). *Setting your Python version*. Recuperado el 29 de octubre de 2026, de https://render.com/docs/python-version

Saravia, E., Liu, H.-C. T., Huang, Y.-H., Wu, J., & Chen, Y.-S. (2018). CARER: Contextualized affect representations for emotion recognition. En *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3687–3697). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1404

Schuster, M., & Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing, 45*(11), 2673–2681. https://doi.org/10.1109/78.650093

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. En *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining* (pp. 29–39).

Anexos

Anexo A. Estructura del proyecto

- app/ — aplicación Flask (rutas, plantillas HTML y estilos)
- src/ — preparación de datos, EDA, modelo, entrenamiento, evaluación, exportación y predicción
- models/ — tokenizador, modelo Keras de inferencia y pesos para NumPy
- reports/ — métricas en JSON, historial de entrenamiento y figuras
- data/sample/ — 60 frases de ejemplo; data/raw/ y data/processed/ se excluyen del repositorio
- tests/ — 20 pruebas automatizadas con pytest
- scripts/smoke_test.py — prueba de humo para la aplicación desplegada
- docs/ — guía de despliegue e informe
- wsgi.py, requirements.txt, requirements-dev.txt, render.yaml, .python-version, README.md

Anexo B. Ejecución reproducible

Con Python 3.13 y las dependencias de *requirements-dev.txt* instaladas, y con el archivo *text.csv* copiado en *data/raw/*, el proyecto completo se reproduce con los siguientes comandos, ejecutados en orden desde la carpeta raíz:

1. python -m src.data_prep — limpieza, deduplicación y división de los datos.
2. python -m src.eda — análisis exploratorio y figuras 1 a 4.
3. python -m src.train — entrenamiento de la red y exportación de los artefactos de inferencia.
4. python -m src.baseline — línea base TF-IDF con regresión logística.

5. python -m src.evaluate — métricas en prueba y figuras de evaluación.
6. python -m pytest — pruebas automatizadas.
7. python wsgi.py — aplicación web en http://127.0.0.1:5000.